

Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky
počítačov a informatiky

Zvýšenie bezpečnosti vybraných šifrovacích algoritmov v prístupových systémoch

Diplomová práca

ACCESS-SECURITY

Systémová príručka

Vedúci diplomovej práce:

doc. Ing. Eva Chovancová, PhD.

Diplomant:

Oleksandr Mykhailyshyn

Konzultant diplomovej práce:

Košice 2026

Contents

1	Funkcia programu	1
2	Analýza riešenia	2
3	Popis programu	2
3.1	Popis riešenia	3
3.2	Popis algoritmov a údajových štruktúr, globálnych premenných . . .	4
3.3	Popis modulov a podprogramov	8
3.4	Popis vstupných a výstupných súborov	14
3.5	Popis správ pre systémového programátora	17
4	Preklad programu	19
4.1	Zoznam zdrojových textov	20
4.2	Požiadavky na technické prostriedky pri preklade	22
4.3	Požiadavky na programové prostriedky pri preklade	22
5	Nadväznosť na iné programové produkty	24
6	Zhodnotenie riešenia	24
7	Zoznam použitej literatúry	26
8	Zoznam príloh	29

List of Tables

3–1 Schéma SQLite databázy	7
3–2 Tok spracovania požiadavky a rozhodovacia pipeline	13
3–3 Vstupné súbory produkčného systému	15
3–4 CLI prepínače experimentálneho harnessu	16
3–5 Výstupy analytického pipeline	18
4–1 Zdrojové súbory serverovej aplikácie (C++20) podľa modulov	21
4–2 Požiadavky na technické prostriedky	22
4–3 Požiadavky na programové prostriedky	23
5–1 Externé produkty a ich rola v systéme	25

1 Funkcia programu

Access-Security je prototyp systému kontroly fyzického prístupu na báze RFID/NFC. Jeho úlohou je autorizovať držiteľov kariet na prechod cez elektronicky riadené dvere na základe kombinácie roly karty a role dverí, pričom celý komunikačný reťazec medzi čítačkou a serverom je chránený kryptograficky — autentifikovaným šifrovaním (AEAD)[13], anti-replay ochranou, HKDF per-reader deriváciou kľúčov[9], detekciou protokolových anomálií[16] a tamper-evident auditom[17].

Systém tvoria tri hlavné časti:

- **Hardvérová čítačka** postavená na mikrokontroléri ESP32[4] a module RC522, komunikujúca podľa ISO 14443[8]. Číta UID karty, pripojí identifikátor čítačky a dverí, doplní monotónne počítadlo `hw_seq` a cez bezdrôtové pripojenie Wi-Fi odošle autentifikovanú HTTP požiadavku (HMAC-SHA-256[11] podpis nad reťazcom „metóda + cesta + LF + telo“) na server.
- **Serverová aplikácia** v jazyku C++20, pozostávajúca z dvoch nezávislých HTTP služieb: *frontend* služba overuje HMAC podpis požiadavky a konštruje interný AEAD frame, *DecisionService* frame dešifruje, overí anti-replay, spustí anomálny detektor, vyhodnotí autorizáciu (RBAC) a výsledok zapíše do auditného logu. Lokálne úložisko je realizované cez SQLite[7] v WAL móde.
- **Webové administrátorské rozhranie** pre správu čítačiek, kariet, rolí dverí, kontrolu auditného logu s overením integrity reťaze a sledovanie prevádzkových udalostí v reálnom čase.

Program je určený na nasadenie v prostredí, kde konvergujú požiadavky fyzickej a informačnej bezpečnosti — tam, kde sa od prístupového systému očakáva ochrana nielen proti jednoduchému skopírovaniu karty, ale aj proti aktívnym útokom

na komunikačný kanál (replay, manipulácia hlavičky, podsunutie nonce, rollback sekvenčného čísla, forgery pokusy o AEAD tag a pod.).

2 Analýza riešenia

Detailný teoretický rozbor problematiky — analýza aktuálnych prístupov ku kontrole fyzického prístupu, model hrozieb (STRIDE[6]), porovnanie kryptografických primitív (ChaCha20-Poly1305[13] vs. AES-GCM, stratégie generovania nonce, HKDF derivácia kľúčov, HMAC autentifikácia), zásady návrhu bezpečných protokolov, ochrana súkromia identifikátorov, tamper-evident auditovateľnosť, bezpečnosť vstavaného zariadenia a bezpečnosť transportnej vrstvy — je obsahom kapitoly 2 *Analýza* hlavnej diplomovej práce, ktorá je súčasťou rovnakého repozitára.

Táto systémová príručka sa zameriava na technickú dokumentáciu konkrétnej implementácie: organizáciu kódovej bázy, algoritmy realizované vlastným kódom, údajové štruktúry, konfiguráciu a spôsob zostavenia a spustenia systému. Pre úplné odôvodnenie zvolených kryptografických primitív a architektonických rozhodnutí je čitateľ odkázaný na hlavnú diplomovú prácu.

3 Popis programu

Program pozostáva z troch vrstiev. Hardvérová vrstva `hw_layer` je firmvér bežiaci na ESP32 s čítačkou MFRC522; jeho úlohou je čítanie UID karty a odoslanie autentifikovanej HTTP požiadavky serveru. Serverová časť je písaná v jazyku C++20 a je rozdelená na dve nezávislé HTTP služby: *frontend* (port 8080), ktorá prijíma požiadavky od čítačiek, overuje HMAC podpis a konštruuje interný AEAD frame, a *DecisionService* (port 8081), ktorá frame dešifruje, spúšťa anti-replay kontrolu, anomálny detektor a RBAC vyhodnotenie a zapisuje výsledok do auditného logu. *DecisionService* počúva výhradne na loopback rozhraní 127.0.0.1,

takže je dostupný len lokálnej frontend službe. Webové administrátorské rozhranie slúži pre správu čítačiek, kariet a rolí dverí a pre sledovanie prevádzky v reálnom čase.

Rozhodovacia cesta (`DecisionEngine::handleFrameBytes`) je organizovaná od najlacnejších kontrol po najdrahšie, aby sa neplatné požiadavky odfiltrovali čo najskôr: parsovanie → validácia čítačky → kontrola verzie kľúča → anti-replay (read-only) → AEAD dešifrovanie → anomálny detektor → RBAC → audit.

3.1 Popis riešenia

Projekt sa skladá z troch komponentov, ktoré dokopy tvoria funkčný prístupový systém spolu s experimentálnou platformou pre jeho vyhodnotenie. Bezpečnostné mechanizmy implementované v serverovej časti (AEAD frame s AAD väzbou, HKDF per-reader derivácia kľúčov, deterministický nonce, anti-replay sliding window, detektor protokolových anomálií s karanténou, tamper-evident audit log a pseudonymizácia identifikátorov kariet) sú podrobne popísané a odôvodnené v kapitole 3 hlavnej diplomovej práce; tu sú spomenuté len v rozsahu potrebnom pre orientáciu v kóde.

Hardvérová vrstva. Firmvér pre ESP32 s čítačkou MFRC522, písaný pre Arduino-ESP32 framework. Číta UID karty, udržiava monotónne počítadlo `hw_seq` v perzistentnom NVS úložisku, a odošle autentifikovanú HTTP požiadavku cez Wi-Fi (HMAC-SHA-256 podpis nad reťazcom "POST /api/hw/uid\n" + telo požiadavky). Detailné zapojenie a popis firmvéru je v následných podkapitolách.

Serverová aplikácia. Dve nezávislé HTTP služby v jazyku C++20: *frontend* (port 8080) prijíma hardvérové požiadavky, overuje HMAC podpis a konštruje interný AEAD frame; poskytuje aj administratívne API a servuje webové rozhranie. *DecisionService* (port 8081, viazaný na loopback) frame dešifruje, spúšťa pipeline kontrol (anti-replay, anomálny detektor, RBAC, audit) a vracia rozhodnutie.

Úložisko je SQLite v režime WAL.

Webové administrátorské rozhranie. Vanilla JavaScript (bez frameworku) servovaný frontend službou. Spravuje čítačky, karty a role dverí, verifikuje integritu auditného logu a sleduje prevádzkové udalosti v reálnom čase (polling každých 500 ms; auditný log auto-refresh každú sekundu).

Experimentálna a analytická vrstva. C++ harness (`experiments/`) zdieľa produkčný kód kryptografickej a rozhodovacej vrstvy, čím sa zaručuje, že merania odrážajú skutočné správanie systému. Obsahuje sedem scenárov pokrývajúcich triedy útokov skúmaných v hlavnej práci. Python skripty (`analysis/`) spracúvajú CSV výstupy a generujú grafy a LaTeX-ready tabuľky pomocou `pandas`, `matplotlib` a `seaborn`.

Vetvenie repozitára. Projekt je udržiavaný v dvoch vetvách: `main` (produkčná, obsahuje len funkčný systém) a `analytical` (rozšírená o moduly `experiments/` a `analysis/`). Rozdiel je čisto aditívny — produkčný kód je identický, `analytical` iba dopĺňa nástroje pre meranie, čím je zaručené, že experimenty bežia na presne tom istom kóde, aký by bol nasadený v produkcii.

3.2 Popis algoritmov a údajových štruktúr, globálnych premenných

Táto podkapitola opisuje vlastné algoritmy implementované v projekte, kľúčové údajové štruktúry, rozdelenie kódu na moduly a model interakcie s používateľom.

Modulárna štruktúra. Serverová časť je rozdelená do deviatich modulov so statickými knižnicami a jednosmernými závislosťami: `crypto_lib` (AEAD, HKDF, generátory nonce, HMAC), `protocol_lib` (binárny formát frame, replay window), `key_manager` (derivácia kľúčov, rotácia), `access_core` (`FrameHandler`,

ProtocolAnomalyDetector), `access_decision` (DecisionEngine, RBAC), `access_storage` (SQLite, audit log), `config_loader` (YAML), `runtime_events` (udalosti pre web) a `access_admin` (HTTP server, služby, web frontend). Každý modul má vlastnú sadu unit testov v podadresári `tests/`.

Algoritmus: rozhodovacia pipeline. Hlavný algoritmus serverovej časti je obsiahnutý v `DecisionEngine::handleFrameBytes` (a podriadenej `FrameHandler::handle`). Spracovanie frame prebieha v pevne definovanom poradí od najlacnejších kontrol po najdrahšie, aby sa neplatné požiadavky odfiltrovali čo najskôr:

1. **Parsovanie frame** — validuje magic, verziu, hlavičku, ciphertext (s kontrolou maximálnej dĺžky pred alokáciou), tag a absenciu trailing bajtov.
2. **Validácia čítačky a verzie kľúča** — overenie, že `reader_id` je registrovaný a `key_version` je aktuálna alebo (pri povolenej tolerancii) bezprostredne predchádzajúca.
3. **Anti-replay kontrola** — *read-only* volanie `ReplayWindow::contains(seq)`; pri nájdení duplicitu sa frame odmietne bez pokusu o dešifrovanie.
4. **AEAD dešifrovanie** — derivácia per-reader kľúča cez HKDF, overenie Poly1305 tagu nad hlavičkou (AAD) a ciphertextom.
5. **Anomálny detektor** — pri úspešnom dešifrovaní vyhodnotenie štyroch protokolových invariantov (viď nižšie).
6. **Zaznamenanie sekvencie (remember)** — až po úspešnom kroku 4 sa `seq` pridá do replay window, aby neplatné frame nemohli „otráviť“ stav.
7. **RBAC autorizácia** — vyhľadanie karty podľa HMAC odtlačku, overenie väzby čítačka $\leftrightarrow$ dvere a povolenia role pre cieľové dvere.

8. **Zápis do auditného logu** — bez ohľadu na výsledok rozhodnutia (allow alebo deny).

Hlavné implementačné algoritmy. `ReplayWindow` kombinuje `std::deque` (FIFO eviction) a `std::unordered_set` (kontrola $O(1)$) plus ukazovateľ `_maxSeen`; tolerujú sa krátke preusporiadania, odmietajú sa duplikáty a hodnoty staršie ako `maxSeen - windowSize`[10]. `HmacNonceGenerator` odvodzuje nonce ako prvých 12, resp. 24 bajtov výstupu HMAC-SHA-256 nad serializovaným kontextom hlavičky[9]; prijímateľ prepočíta hodnotu a porovná v konštantnom čase cez `sodium_memcmp`[3]. `ProtocolAnomalyDetector` udržiava per-reader stav a zakaranténuje čítačku pri replay, nonce mismatch, sekvenčnom rollbacku alebo `tag_fail_streak`[12, 16]. `SqliteAuditLog` reťazí záznamy cez `HMAC(K_audit, prev_hash || fields)` a paralelne aktualizuje `audit_anchor`[17]. `CardIdHasher` ukladá UID len ako HMAC s pepperom; pri rotácii sa hľadanie skúša s aktuálnym aj predchádzajúcim pepperom. Parser binárneho frame validuje magic `AS01`, verziu, dĺžku ciphertextu (limit pred alokáciou) a absenciu trailing bajtov; každá neplatná pozícia končí výnimkou s kódom chyby. Detailné matematické formulácie a odôvodnenia sú v kapitole 3 hlavnej diplomovej práce.

Konfigurácia. Behová konfigurácia je udržiavaná v YAML súboroch v adresári `config/`. Hlavný produkčný profil `access_security.yaml` obsahuje štyri skupiny parametrov (`frame_handler`, `key_management`, `storage`, `admin`). Šesť experimentálnych profilov `profile_a{1,2}_r{0,1,2}.yaml` tieto sekcie dopĺňa o piatu skupinu `experiment` (cipher mode, nonce mode, detektor), čím kombinujú dva AEAD algoritmy a tri stratégie generovania nonce.

Modul `config_loader` poskytuje funkciu `loadFromYaml(path) → Config`, ktorá načíta YAML a validuje hodnoty. Výsledná štruktúra `Config` agreguje päť podštruktúr: `FrameHandlerConfig` (anti-replay, max skew, väzby), `StorageConfig` (cesta k SQLite), `AdminConfig` (HTTP port, admin token, HMAC secret,

decision_mode), KeyManagementYaml (verzia kľúča, cesta k master key) a ExperimentConfig (AEAD cipher, nonce stratégia, detektor, kotva auditu). Všetky polia majú rozumné predvolené hodnoty, takže konfigurácia je priamočiara a zrozumiteľná bez dokumentácie.

Schéma databázy. Úložisko je realizované cez SQLite v režime WAL. Schéma obsahuje šesť tabuliek (tabuľka 3–1), ktoré sú pri prvom spustení automaticky vytvorené modulom access_storage cez CREATE TABLE IF NOT EXISTS.

Table 3–1 Schéma SQLite databázy

Tabuľka	Obsah
cards	HMAC(pepper, UID) → rola karty
door_roles	Povolené roly pre každé dvere (RBAC)
readers	Registrované čítačky s verziou kľúča
reader_doors	Väzba čítačka ↔ dvere (FK, CASCADE)
audit_log	Záznamy auditu s HMAC hash chain
audit_anchor	Ochranná kotva proti truncation

Žiadna tabuľka neobsahuje surové UID kariet — ukladá sa len HMAC odtlačok s pepperom. Tabuľky audit_log a audit_anchor spolu tvoria tamper-evident auditný mechanizmus opísaný v podkapitole 3.1.

Globálne premenné. Projekt sa zámerne vyhýba globálnym mutable premenným. Jediný agregujúci objekt serverovej aplikácie je inštancia AppState (v module access_admin), ktorá drží zdieľaný stav — DecisionEngine, KeyManager, úložisko, kruhový buffer prevádzkových udalostí — a dva mutexy: app.m pre administratívne API a app.engineMutex pre rozhodovaciu cestu. Všetky modifikácie zdieľaného stavu sú chránené jedným z týchto mutexov. Firmvérová časť udržiava len statické premenné v rámci hlavného skechu (hwSeq, lastUid, lastSendMs) — plne opísané sú v následnej podkapitole o hardvérovej vrstve.

Dialóg s používateľom. Interakcia s administrátorom prebieha výhradne cez webové rozhranie: používateľ v prehliadači otvorí administrátorskú stránku (servovanú frontend službou na porte 8080), zadá voliteľný administrátorský token a prepína medzi sekciami *Readers*, *Door roles*, *Cards*, *Audit*, *Runtime logs* a *DB Export/Import*. Každá operácia (pridanie karty, zrušenie roly, verifikácia auditu) je realizovaná volaním REST endpointu; výsledok je zobrazený v zodpovedajúcej tabuľke alebo stavovom paneli. Dialóg s používateľom karty (t. j. samotné priloženie karty k čítačke) je realizovaný cez svetelnú a zvukovú signalizáciu opísanú v následnej podkapitole o hardvérovej vrstve.

3.3 Popis modulov a podprogramov

Táto podkapitola sumarizuje verejné rozhranie každého modulu — najdôležitejšie triedy a funkcie, ich vstupno-výstupné parametre a vzťah k ostatným modulom. Globálne premenné jednotlivé moduly nepoužívajú; všetok zdieľaný stav je centralizovaný v objekte `AppState` (popísaný v 3.2). Poradie popisu zodpovedá závislostiam: od základných knižníc po vyššie vrstvy.

hw_layer (firmvér ESP32). Súbor `hw_layer.ino` obsahuje obvyklú dvojicu Arduino funkcií. `setup()` vykoná inicializáciu (sériová konzola, GPIO, LEDC, NVS, SPI, MFRC522, Wi-Fi). `loop()` periodicky volá `rfid.PICC_IsNewCardPresent()` a `rfid.PICC_ReadCardSerial()`; pri detekcii karty zavolá `postUidToServer(uid, allow)`. Tá overí Wi-Fi, inkrementuje `hw_seq`, zostaví JSON telo, vypočíta HMAC podpis (`hmacSha256Hex`) a odošle požiadavku cez `HTTPClient`. Výsledok je signalizovaný cez `signalAllow()` alebo `signalDeny()` (LED + LEDC tón).

crypto_lib. Trieda `SecureAead` (`secure_aead.hpp`) zapuzdruje ChaCha20-Poly1305 alebo XChaCha20-Poly1305 z knižnice `libsodium`. Verejné šifrovacie metódy: `sealWithSeq(plaintext, aad, seq)` (interne odvodí nonce zo `seq`) a `sealWithNonce(plaintext, aad, nonce)` (explicitne dodaný nonce) — obe

vracajú `Ciphertext` obsahujúci `ciphertext`, `tag` a `nonce`. Dešifrovanie je metódou `openWithNonce(ct, tag, aad, nonce)`, ktorá vracia `std::vector<uint8_t>` s plaintextom alebo vyhodí výnimku pri zlyhaní tag verifikácie. Trieda `Hkdf` (`hkdf.hpp`) implementuje HKDF-SHA-256 podľa RFC 5869: konštruktor vykoná extract fázu ($PRK = \text{HMAC}(\text{salt}, \text{IKM})$), metóda `derive()` $\rightarrow \text{vector<uint8_t>}$ expand fázu. Generátory nonce (`nonce_generator.hpp`) implementujú spoločné rozhranie `INonceGenerator::generate(context, seq) \rightarrow \text{array<uint8_t, 24>}`; konkrétne triedy `RandomNonceGenerator` (`libsodium randombytes_buf`) a `HmacNonceGenerator` (HMAC nad kontextom).

protocol_lib. `Frame` (`frame.hpp`) je dátová štruktúra obsahujúca `Header`, `ciphertext` (`vector<uint8_t>`) a `Tag16`. Voľná funkcia `serialize(const Frame&) \rightarrow \text{vector<uint8_t>}` vyprodukuje binárny tvar (`magic`, `verzia`, `hlavička`, `dĺžka ciphertext`, `ciphertext`, `tag`); `parseFrame(bytes, maxCtLen=4096) \rightarrow Frame` robí opačnú operáciu cez triedu `FrameParser` s ochranou proti DoS útoku. Trieda `ReplayWindow` (`replay_window.hpp`) poskytuje `contains(seq) \rightarrow bool`, `remember(seq)` a `accept(seq) \rightarrow bool` (atomická kontrola + vloženie).

key_manager. Trieda `KeyManager` (`key_manager.hpp`) odvodzuje podkľúče z master kľúča. Konštruktor prijíma 32-bajtový *MasterKey* a *KeyManagerConfig* (aktuálna verzia, povolenie predošlej verzie). Verejné metódy:

- `deriveAeadKey(readerId, keyVersion)` — HKDF AEAD kľúč so saltom obsahujúcim ID čítačky a verziu,
- `deriveNonceKey(readerId, keyVersion)` — per-reader kľúč pre HMAC nonce konštrukciu,
- `deriveAuditHmacKey()` — per-inštanciu kľúč pre auditný hash chain (bez parametrov),
- `deriveCardPepper(keyVersion)` — pepper pre pseudonymizáciu kariet,

- `isAcceptedKeyVersion(keyVersion)` — kontrola, či je verzia akceptovaná pre dešifrovanie,
- `masterAsAeadKey()` — priamy režim bez HKDF (pre testovanie).

access_core. `FrameHandler::handle(frameBytes) → HandleResult` je centrálna metóda spracovania frame: parsuje, validuje čítačku a verziu kľúča, volá `ReplayWindow::contains`, deriváciu kľúča a AEAD dešifrovanie. `HandleResult` obsahuje pole `allow`, `reason` (string code), `plaintext` a `header`. `ProtocolAnomalyDetector` (`protocol_anomaly_detector.hpp`) ponúka per-reader päť report metód: `reportReplay`, `reportSeq`, `reportNonceMismatch`, `reportTagFailure` a `reportSuccess`; spolu s dotaznými `isQuarantined` a `quarantineInfo`. Trieda je *thread-safe* (interný mutex).

access_decision. Hlavná metóda `handleFrameBytes` (s parametrami `frameBytes` a mapou replay window podľa čítačky) vracia `DecisionResult` a orchestruje úplný rozhodovací cyklus: volá `FrameHandler::handle`, vyhodnotí stav anomálneho detektora, prevedie pseudonymizáciu UID cez `CardIdHasher::hmacHex`, vykoná RBAC vyhľadanie cez `IAccessStore`, zapíše do `IAuditLog` a vyšle udalosť do voliteľnej `EventBus`. Štruktúra `DecisionResult` obsahuje `allow`, `reason` (kódy ako `ok`, `replay`, `decrypt_failed`, `unknown_card`, `forbidden`, `quarantined`). `CardIdHasher::hmacHex(uid) → string` vypočíta hex HMAC s pepperom.

access_storage. `SqliteAccessStore` implementuje rozhranie `IAccessStore` s CRUD operáciami nad tabuľkami `readers`, `cards`, `door_roles`, `reader_doors`. `SqliteAuditLog` implementuje `IAuditLog::append(event)`; interne pri zápise volá `getLastEntryHash()` a `computeEntryHash(prev, event)` pre tvorbu HMAC chain a aktualizuje tabuľku `audit_anchor`. Verifikácia (`verifyAuditChain(db, key)` v `audit_verify.cpp`) prečíta všetky záznamy v poradí a rekonštruuje reťazec;

vracia `AuditVerifyResult {bool ok, int64_t bad_id, std::string error}`
— pri prvej zistenej nezhode sa verifikácia zastaví a vráti ID problémového záznamu spolu s popisom chyby.

Pomocné moduly. Modul `config_loader` obsahuje jedinú verejnú funkciu `loadFromYaml(path)`, ktorá načíta YAML súbor cez `yaml-cpp` a vráti naplnenú štruktúru `Config`. `runtime_events` poskytuje triedu `EventBus` s kruhovým bufferom prevádzkových udalostí (`push(event)`, `getAfter(afterId, limit)`) pre real-time zobrazovanie vo webovom rozhraní. `access_admin` obsahuje vstupný bod `main()` (spustí dva HTTP servery), centrálny `AppState` a service vrstvu (`hw_service`, `cards_service`, `readers_service`, `audit_service` a ďalšie), ktorá tvorí tenkú adaptáciu medzi HTTP požiadavkami a jadrom systému.

Webové rozhranie (web/). Vanilla JavaScript priamo servovaný frontend službou. Modul `app.js` obsluhuje navigáciu medzi sekciami a spúšťa `startEventPolling()` (interval 500 ms) a `startAuditAutoRefresh()` (interval 1 s, aktívny iba pri otvorenej audit sekcii). Jednotlivé súbory `readers.js`, `cards.js`, `doors.js`, `audit.js`, `db.js` implementujú obslužné funkcie pre svoje sekcie; všetky volajú jednotné API helper `api(path, opts)` z `utils.js`, ktoré pridá `X-Admin-Token` hlavičku.

experiments (harness a scenáre). Samostatný CMake podprojekt mimo produkčného runtime, ktorý linkuje produkčné knižnice ako spotrebiteľ. Skladá sa zo statickej knižnice `experiments_common` a ôsmich samostatných spustiteľných programov (`s1_replay` – `s7_nonce_tamper`). Verejné API tvoria: trieda `ExperimentContext` (`experiment_context.hpp`), ktorá pre každú iteráciu inšancuje vlastnú kópiu rozhodovacej pipeline (`KeyManager`, `SqliteAccessStore`, `DecisionEngine`, `ProtocolAnomalyDetector`) a vystavuje metódu `processFrame(frameBytes) → DecisionResult` s voliteľným E2E režimom cez `setE2eUrl(url)`; abstraktná trieda `IScenario`

v `scenario_runner.hpp` s metódami `setup(ctx)` a `buildTrialFrame(ctx)`, ktoré každý scenár implementuje; a voľná funkcia `runScenario(scenario)` iterujúca cez profily, kroky a behy. Orchestráciu E2E režimu zabezpečuje shell skript `run_e2e.sh`, ktorý pre každý profil generuje dočasný YAML konfig, spúšťa reálny `access_admin` server, seeduje databázu cez REST API a agreguje CSV výstupy.

analysis (Python pipeline pre grafy a tabuľky). Samostatný Python modul mimo CMake buildu, ktorý spracúva CSV výstupy harnessu a generuje publikačné grafy aj LaTeX tabuľky pre prácu. Vstupným bodom je skript `analyze.py` s CLI prepínačmi `-results-dir` (predvolene `build/results`) a `-output-dir` (predvolene `analysis/plots`). Závislosti sú uvedené v `requirements.txt`: `pandas>=2.0` (spracovanie CSV), `matplotlib>=3.7` a `seaborn>=0.13` (grafy), `numpy>=1.24` (numerické výpočty). Globálne konštanty v hornej časti skriptu (`PROFILE_ORDER`, `PROFILE_PALETTE`, `MODE_LABELS`, `ALGO_LABELS`, `SCENARIO_TITLES`) garantujú konzistentné farby, poradie a popisy profilov naprieč všetkými výstupmi.

Pipeline produkuje dve kategórie výstupov do `analysis/plots/`: (a) sedem PNG grafov — `attack_success_heatmap`, `attack_success_by_n`, `quarantine_speed`, `latency_boxplot`, `throughput`, `detector_overhead` a `scenario_summary_table`, všetky uložené pri 300 DPI pre tlač; (b) tri LaTeX tabuľky — `scenario_summary.tex`, `throughput_table.tex` a `baseline_correctness.tex`, ktoré sa vkladajú priamo do textu diplomovej práce cez `\input{plots/...}`. Tento mechanizmus zabezpečuje, že numerické hodnoty v tabuľkách sú vždy generované z najnovších experimentálnych dát a nemôžu sa „rozísť“ s grafmi.

Tok spracovania požiadavky. Tabuľka 3–2 zachytáva cestu požiadavky od priloženia karty cez tri vrstvy systému (`ESP32` → `frontend` → `DecisionService`) až po návrat rozhodnutia firmvéru, vrátane vnútornej štruktúry rozhodovacej pipeline.

Table 3 – 2 Tok spracovania požiadavky a rozhodovacia pipeline

#	Vrstva / volanie	Účel
<i>ESP32 firmvér</i>		
1	PICC_ReadCardSerial()	Detekcia karty, čítanie UID.
2	postUidToServer(uid, allow)	Inkrementácia hw_seq (NVS), HMAC podpis, HTTP POST.
<i>Frontend (port 8080)</i>		
3	verifyHwSignature(...)	Overenie HMAC podpisu požiadavky.
4	buildEncryptedFrameBytes(...)	Nonce, AEAD šifrovanie, serializácia frame.
5	POST /api/decision/frame →	HTTP POST na loopback 127.0.0.1:8081.
<i>DecisionService (port 8081, loopback) — handleFrameBytes</i>		
6	parseFrame(bytes, maxCtLen)	Magic, verzia, dĺžka (limit 4096 B), trailing bajty.
7	validateReader / KeyVersion	Registrácia čítačky a akceptovaná verzia kľúča.
8	ReplayWindow::contains(seq)	Anti-replay (read-only).
9	deriveAeadKey + openWithNonce	HKDF per-reader + AEAD dešifrovanie.
10	ReplayWindow::remember(seq)	Zápis sekvencie — až po úspešnom dešifrovaní.
11	isQuarantined / reportSeq	R2: kontrola karantény a rollbacku.
12	CardIdHasher + roleForCardHmac	Pseudonymizácia UID, vyhľadanie role.
13	isAllowed(doorId, role)	RBAC: väzba dvere ↔ rola.
14	IAuditLog::append + EventBus	HMAC hash chain + udalosť pre web.
<i>Návrat odpovede</i>		
15	← {"allow": true false}	JSON odpoveď firmvéru.
16	signalAllow / signalDeny	Svetelná a zvuková signalizácia.

Vzťah modulov navzájom. Smer závislostí v produkčnom runtime je striktné jednosmerný: `access_admin` a `access_decision` sú vyššie vrstvy a používajú nižšie (`access_core`, `access_storage`, `key_manager`, `protocol_lib`, `crypto_lib`). Žiadny nižší modul nezávisí od vyššieho, čím je zaručená testovateľnosť každého modulu izolovane (každý má vlastný adresár `tests/`). `config_loader` a `runtime_events` sú „laterálne“ pomocné moduly bez závislostí na rozhodovacej ceste. Modul `experiments` je voči produkčnému runtime paralelný — linkuje produkčné knižnice ako spotrebiteľ, ale nie je nimi linkovaný; vďaka tomu ich môže používať bez rizika, že sa experimentálny kód dostane do produkčného binárneho výstupu. Modul `analysis` je úplne oddelený od C++ ekosystému (Python) a s ostatnými časťami komunikuje len cez CSV súbory na disku.

3.4 Popis vstupných a výstupných súborov

Táto podkapitola sumarizuje vstupné a výstupné súbory produkčného systému a experimentálnej platformy. Produkčné vstupy sú v tabuľke 3–3; produkčné výstupy sú popísané v nasledujúcom odseku. Experimentálna časť (tabuľky 3–4–3–5) pokrýva CLI parametre, CSV výstupy harnessu a výstupy analytického pipeline.

Produkčné vstupy. Systém vyžaduje tri vstupné súbory (tabuľka 3–3): `config/access_security.yaml` (jediný súbor upravovaný pri nasadení), `secrets/master_key.hex` (jednorázovo generovaný, mimo repozitára) a `hw_layer/hw_config.h` (kompiluje sa priamo do firmvéru).

Produkčné výstupy. Pri prevádzke systém produkuje: `data/access.db` — SQLite databázu (WAL mód) so 6 tabuľkami (`cards`, `door_roles`, `readers`, `reader_doors`, `audit_log` s HMAC hash chain, `audit_anchor` proti truncation; schéma v tabuľke 3–1), automaticky vytvorenú cez `CREATE TABLE IF NOT EXISTS`; JSON HTTP odpovede vo formáte `{"allow": bool, "reason": string}` (HW endpoint) a `{"status": "ok"|"error", ...}` (admin API, autentifikácia cez

Table 3 – 3 Vstupné súbory produkčného systému

Súbor	Formát	Obsah
config/access_security.yaml	YAML	Hlavný konfiguračný súbor so štyrmi sekciami: frame_handler (anti-replay, max skew, AAD mód), key_management (verzia kľúča, cesta k master key), storage (cesta k SQLite), admin (port, token, HMAC secret, decision mode). Sekcia experiment je len v 6 profilových YAML súboroch.
secrets/master_key.hex	Text (hex)	64 hexadecimálnych znakov (32 bajtov po dekódovaní). Zdieľané tajomstvo pre HKDF deriváciu všetkých podkľúčov. Nie je súčasťou repozitára.
hw_layer/hw_config.h	C hlavička	Konfigurácia firmvéru: Wi-Fi prístupové údaje, URL servera, HMAC secret, identita čítačky, priradenie GPIO pinov, časové konštanty (popísané v podkapitole 3.1).

X-Admin-Token); voliteľný debug výstup na sériovej konzole ESP32 (UID, HMAC podpis, HTTP kód — v produkcii odporúčané vypnúť).

Vstupné súbory experimentov. Experimentálny harness nevyžaduje YAML konfiguráciu — šesť profilov je definovaných programaticky vo funkcii `allProfiles()`, čím sa eliminuje závislosť na externých súboroch a riziko nekonzistencie medzi konfiguráciou a kódom. Produkčný `master_key.hex` sa načíta len v E2E režime (pri in-process behu harness používa deterministický kľúč `0x00..0x1F` pre reprodukovateľnosť). Všetky parametre behu sú riadené cez CLI prepínače uvedené v tabuľke 3–4.

Table 3–4 CLI prepínače experimentálneho harnessu

Prepínač	Predvolená hodnota	Význam
<code>-seed=N</code>	42	Základ pre seed behov (42 + <code>run_idx</code>).
<code>-warmup=N</code>	200	Počet warmup frame (nezaznamenávaný).
<code>-baseline=N</code>	200	Počet baseline frame (validné, zaznamenávané).
<code>-runs=N</code>	5	Počet opakovaní pre štatistickú robustnosť.
<code>-e2e=URL</code>	—	E2E režim: frame sa posiela cez HTTP na bežiaci server.
<code>-profile=P</code>	—	Filter na jeden profil (napr. A1–R2).

Šesť experimentálnych YAML profilov (`config/profile_a{1,2}_r{0,1,2}.yaml`) slúži výhradne pre E2E režim (skript `run_e2e.sh`).

Výstupné súbory experimentov. Každý zo 8 scenárov produkuje jeden CSV súbor s názvom zhodným so scenárom (`s1_replay.csv` až `s7_nonce_tamper.csv` vrátane variantu `s3_cross_reader.csv`) do adresára

`build/experiments/results/` (in-process) alebo `build/experiments/results_e2e/` (E2E režim). CSV formát bol zvolený pre priamu kompatibilitu s analytickým pipeline (`pandas.read_csv`) a možnosť manuálnej inšpekcie v tabuľkovom procesore.

Každý CSV súbor má jednotnú štruktúru s 15 stĺpcami: `profile`, `cipher_mode`, `nonce_mode`, `detector_enabled` (konfigurácia behu); `scenario`, `phase` (`baseline/attack/measure`), `attack_n`, `frame_idx` (kontext); `allowed`, `reason` (kódy ako `ok`, `replay`, `quarantined`), `quarantined`, `anomaly_type` (rozhodnutie); `latency_ns` (meranie); `seed`, `run_idx` (reprodukovateľnosť). Celkový objem dát pri štandardnom behu (5 behov, `attack_n` = 50...500): ~2,835,000 riadkov naprieč 8 CSV súbormi.

Výstupy analytického pipeline. Skript `analysis/analyze.py` načíta CSV súbory a vygeneruje do adresára `analysis/plots/` dve kategórie výstupov — sedem PNG grafov a tri L^AT_EX tabuľky — zhrnuté v tabuľke 3–5.

LaTeX tabuľky sú generované z rovnakých dát ako grafy a vkladajú sa priamo do textu práce cez `\input{plots/...}`, čím je zaručená konzistencia numerických hodnôt naprieč grafmi a tabuľkami.

3.5 Popis správ pre systémového programátora

Serverová aplikácia. Server vypisuje na štandardný výstup prevádzkové správy:

- **Admin server:** `http://0.0.0.0:8080` — frontend služba úspešne nasadená.
- **Decision service:** `http://127.0.0.1:8081` — DecisionService úspešne nasadený.
- **listen failed** — nepodarilo sa naviazať port (typicky obsadený iným procesom alebo nedostatočné oprávnenia).

Table 3 – 5 Výstupy analytického pipeline

Súbor	Typ	Obsah
attack_success_heatmap.png	PNG (300 DPI)	Heatmapa úspešnosti útokov (scenár $\times$ profil).
attack_success_by_n.png	PNG	Závislosť úspešnosti od počtu frame.
quarantine_speed.png	PNG	Počet frame do prvej karantény (R2).
latency_boxplot.png	PNG	Distribúcia latencie (baseline vs. attack).
throughput.png	PNG	Priepustnosť a percentily latencie (S6).
detector_overhead.png	PNG	Porovnanie R1 vs. R2 na baseline.
scenario_summary_table.png	PNG	Súhrnná tabuľka výsledkov.
scenario_summary.tex	L ^A T _E X	Tabuľka výsledkov (vkladaná cez <code>\input</code>).
throughput_table.tex	L ^A T _E X	Výkonnostné metriky (FPS, percentily).
baseline_correctness.tex	L ^A T _E X	False reject rate na baseline.

- `[decision] exception: <popis>` — výnimka pri spracovaní frame; frame je zamietnutý s dôvodom `internal_error`, server pokračuje v prevádzke.

Detailné prevádzkové udalosti (rozhodnutia o prístupe, detekované anomálie, aktivácie karantény) nie sú vypisované na štandardný výstup — sú prístupné cez webové administrátorské rozhranie v sekciách *Runtime logs* (polling 500 ms) a *Audit* (auto-refresh 1 s). Chybové kódy vrátené v JSON odpovediach na HW endpoint (`reason: ok, replay, decrypt_failed, unknown_card, forbidden, quarantined, bad_payload, bad_action, remote_unavailable, internal_error`) sú popísané v používateľskej príručke.

Firmvér ESP32. Firmvér vypisuje na sériovú konzolu (115 200 baud) správy pri každej fáze spracovania: inicializácia (`hw_seq loaded, RC522 ready`), pripojenie k Wi-Fi (`Connecting...`, `OK/FAILED`), detekcia karty (`UID: <hex>`), odoslanie požiadavky (`POST <URL>, Body: ..., Sig: ...`), odpoveď servera (`HTTP <kód>, Resp: ...`) a finálne rozhodnutie (`ALLOW/DENY/SEND FAILED`). Chybové stavy: `WiFi not connected` (zlyhanie Wi-Fi), `Skip (too soon)` (debounce 1500 ms), `Bad response JSON (no allow field)` (neplatná odpoveď servera). Sériový výstup obsahuje citlivé údaje (UID, HMAC podpis) — v produkčnom nasadení sa odporúča vypnúť.

4 Preklad programu

Systém sa skladá z troch nezávisle zostavovaných častí: serverovej aplikácie (C++20, CMake), firmvéru čítačky (Arduino/ESP32) a analytického pipeline (Python). Táto kapitola opisuje štruktúru prekladového procesu, potrebné závislosti a spôsob zostavenia každej časti. Konkrétne príkazy na zostavenie a spustenie sú uvedené v používateľskej príručke.

Serverová aplikácia (C++20). Koreňový `CMakeLists.txt` so štandardom C++20 registruje deväť produkčných modulov (`crypto_lib` až `access_admin`) a modul `experiments`. Každý modul je statická knižnica; `access_admin` navyše produkuje hlavný binárny súbor servera, scenáre v `experiments/` sú samostatné spustiteľné programy. CMake `FetchContent` pri prvom zostavení automaticky stiahne 5 externých knižníc (konfigurácia v `cmake_other/third_party.cmake`). Predpoklady a verzie sú v tabuľke 4–3. Verifikácia: `ctest` (44 testov, voliteľne ASan/UBSan cez CMake príznaky).

Docker nasadenie. Multi-stage Dockerfile: `builder` (`ubuntu:24.04` + `build-essential`, `cmake`, `g++`, `libsqlite3-dev`, `python3`, `git`) skompiluje projekt a spustí testy; `runtime` (`ubuntu:24.04` + `libsqlite3-0`) obsahuje len binárny súbor servera, web rozhranie a konfiguráciu. Adresáre `data/` a `secrets/` sa pri spustení pripájajú ako bind mounts (`-v`).

Firmvér ESP32 a Python pipeline. Firmvér sa kompiluje v Arduino IDE 2.x (vyžaduje Espressif Arduino Core a knižnicu MFRC522, úpravu `hw_config.h` a nahratie cez USB). Analytický pipeline vyžaduje Python ≥ 3.10 a knižnice z `analysis/requirements.txt`; vstupom je skript `analyze.py`, výstupom PNG grafy a LaTeX tabuľky v `analysis/plots/` (tabuľka 3–5).

4.1 Zoznam zdrojových textov

Kompletné zdrojové texty sú súčasťou priloženého média. Tabuľka 4–1 sumarizuje moduly serverovej aplikácie; ostatné komponenty (firmvér, web, experimenty, analýza) sú zhrnuté v nasledujúcom odseku.

Ostatné komponenty: `experiments/` — 20 súborov C++ (6 spoločných + 8 scenárov + 6 hlavičiek); `hw_layer/` — 2 súbory firmvéru ESP32 (`.ino`, `.h`); `web/` — 10 súborov frontendu (HTML, 8 JS, CSS); `analysis/` — 1 Python skript (`analyze.py`); `config/` — 7 YAML konfigurácií (1 produkčná + 6 profilov);

Table 4–1 Zdrojové súbory serverovej aplikácie (C++20) podľa modulov

Modul	.hpp	.cpp	Testy	Hlavná zodpovednosť
crypto_lib	4	4	2	AEAD, HKDF, HMAC, nonce generátory
protocol_lib	5	5	3	Binárny formát frame, replay window
key_manager	2	2	0	HKDF derivácia, hex utility
access_core	3	3	2	FrameHandler, anomálny detektor
access_decision	6	4	2	DecisionEngine, RBAC, CardIdHasher
access_storage	5	4	2	SQLite, audit log, verifikácia chain
config_loader	1	1	0	Načítanie YAML konfigurácie
runtime_events	1	1	0	EventBus (udalosti pre web)
access_admin	12	13	0	HTTP server, služby, main()
Spolu	39	37	11	76 zdrojových + 11 testovacích súborov

16 CMakeLists.txt, Dockerfile a run_e2e.sh.

4.2 Požiadavky na technické prostriedky pri preklade

Táto podkapitola uvádza požiadavky na hardvér; požiadavky na programové vybavenie sú v podkapitole 4.3. Tabuľka 4–2 sumarizuje minimálne a odporúčané parametre.

Table 4–2 Požiadavky na technické prostriedky

Parameter	Preklad (build)	Prevádzka (runtime)
Procesor	x86-64 alebo ARM64; min. 2 jadrá (4+ pre paralelný preklad)	x86-64 alebo ARM64; 1+ jadro
Operačná pamäť	Min. 2 GB (4 GB odporúčané)	Min. 128 MB
Diskový priestor	~1 GB (zdrojový kód 35 MB + závislosti 325 MB + build 500 MB)	~150 MB (Docker obraz 123 MB)
Sietové pripojenie	Áno (prvé zostavenie stiahne závislosti)	Wi-Fi pre komunikáciu s čítačkou
Prídavné zariadenia	—	ESP32 DevKit + MFRC522 + USB kábel

Pre preklad firmvéru ESP32 postačuje počítač s ~1 GB voľného miesta (Arduino IDE + ESP32 toolchain) a USB portom. Analytický pipeline (Python) vyžaduje ~200 MB pre virtuálne prostredie a min. 1 GB RAM pre spracovanie ~3M riadkov CSV. Pri použití Docker nasadenia sú požiadavky na preklad aj prevádzku pokryté kontajnerom — stačí nainštalovaný Docker Engine.

4.3 Požiadavky na programové prostriedky pri preklade

Tabuľka 4–3 sumarizuje programové vybavenie potrebné pre zostavenie a prevádzku jednotlivých častí systému.

Table 4 – 3 Požiadavky na programové prostriedky

Komponent	Nástroj / knižnica	Verzia
<i>Serverová aplikácia (preklad)</i>		
Operačný systém	Linux (Ubuntu 22.04+)	—
C++ kompilátor	GCC (podpora C++20)	≥ 12
Build systém	CMake	≥ 3.20
Databáza (hlavičky)	SQLite3 (<code>libsqlite3-dev</code>)	3.x
Závislosti (auto)	libsodium, nlohmann/json, yaml-cpp, cpp-httpplib, Google Test	FetchContent
<i>Serverová aplikácia (prevádzka)</i>		
Operačný systém	Linux (Ubuntu 22.04+); Docker	—
Databáza (runtime)	SQLite3 (<code>libsqlite3-0</code>)	3.x
Kontajner (voliteľ.)	Docker Engine	20.10+
<i>Firmvér čítačky</i>		
Vývojové prostredie	Arduino IDE (Linux)	2.x
Podpora platformy	Espressif Arduino Core (ESP32)	3.x
Knižnica RFID	MFRC522 (Miguel Balboa)	1.4.x
<i>Analytický pipeline</i>		
Interpreter	Python	≥ 3.10
Knižnice	pandas, matplotlib, seaborn, numpy	<code>req.txt</code>

Všetky C++ závislosti servera sa získavajú automaticky cez `CMake FetchContent`; nie je potrebný žiadny externý správca balíkov. Python závislosti sa inštalujú cez `pip` z priloženého `requirements.txt`.

5 Nadväznosť na iné programové produkty

Program *Access-Security* stojí na viacerých externých open-source produktoch s permissívnymi licenciami (ISC, MIT, BSD, Apache 2.0). Tabuľka 5–1 sumarizuje ich rolu v systéme; konkrétne verzie sú uvedené v tabuľke 4–3 (podkapitola 4.3). Kryptografická knižnica `libsodium`[3] prešla nezávislým bezpečnostným auditom[14] a všetky jej operácie sú implementované v konštantnom čase[2]. Na ESP32 je `mbedtls` jedinou dostupnou implementáciou HMAC v rámci Arduino Core.

6 Zhodnotenie riešenia

Implementácia spĺňa hlavný cieľ práce — funkčný prototyp prístupového systému s viacvrstvovou kryptografickou ochranou, experimentálne overený v siedmich scenároch. Nasledujúce body sumarizujú aktuálne obmedzenia, nedopracované oblasti a potenciálne smery ďalšieho vývoja.

Obmedzenia produkčného nasadenia. Komunikácia prebieha cez nešifrovaný HTTP — HMAC zabezpečuje integritu a autentifikáciu, ale nie dôvernosť (pasívny pozorovateľ vidí UID); nasadenie `mTLS`[15] by túto medzeru uzavrelo. Master kľúč je v hex súbore a HMAC secret firmvéru je vkompileovaný do binárky — pre produkciu vhodnejšie HSM alebo vault[1]. Porovnanie admin tokenu nie je konštantnočasové (teoretický timing side-channel pri krátkych tokenoch). Karta slúži ako identifikátor, nie kryptografický dôkaz identity — ochrana proti klonovaniu je len na serverovej strane (anti-replay, anomálny detektor); migrácia na MIFARE DESFire[5] by túto vrstvu posilnila.

Table 5 – 1 Externé produkty a ich rola v systéme

Produkt	Rola	Spôsob integrácie
<i>Server (C++20)</i>		
libsodium	AEAD, HMAC, HKDF, randombytes, constant-time compare	FetchContent (zdrojový kód)
SQLite3	Relačná DB (WAL) pre čítačky, karty, audit	Systémová knižnica
cpp-httplib	HTTP server (frontend + DecisionService)	Header-only, FetchContent
nlohmann/json	JSON parsovanie/serializácia	Header-only, FetchContent
yaml-cpp	Načítanie YAML konfigurácie	Statická, FetchContent
Google Test	Unit testy (44 prípadov)	Statická, FetchContent
<i>Firmvér ESP32</i>		
Arduino Core	HAL: Wi-Fi, NVS, GPIO, SPI, LEDC	Arduino Board Manager
MFRC522	Ovládač RFID čítačky (ISO 14443)	Arduino Library Manager
mbedtls	HMAC-SHA-256 pre autentifikáciu	Súčasť ESP-IDF
<i>Analytický pipeline a nasadenie</i>		
pandas, numpy, matplotlib, seaborn	Spracovanie CSV, grafy, percentily	pip (requirements.txt)
Docker Engine	Kontajnerizácia (multi-stage)	Systémový balík

Nedopracované oblasti. Automatická rotácia kľúčov podľa časového plánu (manuálna cez `key_version` podporovaná). Offline režim na čítačke (aktuálne fail-closed pri výpadku servera). Sériový debug výstup ESP32 vypisuje citlivé údaje — v produkcii je potrebné vypnúť. Karanténa nemá voliteľný časový limit pre automatické zrušenie.

Potenciálne smery ďalšieho vývoja. Geograficky distribuované nasadenie cez synchronizáciu stavu (replay window, karanténa) medzi inštanciami DecisionService. Adaptívna úprava prahov detektora (`rollbackThreshold`, `tagFailStreakLimit`) podľa štatistík prevádzky. Rozšírenie experimentálnej sady o adaptívneho útočníka, ktorý striedaním platných a neplatných frame obchádza `tag_fail_streak`. Doplnenie webového rozhrania o autentifikáciu používateľov, auditovanie admin akcií a HTTPS.

Celkové zhodnotenie. Napriek uvedeným obmedzeniam prototyp demonštruje, že kombinácia štandardných kryptografických primitív (ChaCha20-Poly1305[13], HMAC-SHA-256[11], HKDF[9]) s premyslenými architektonickými rozhodnutiami (deterministický nonce, runtime anomálna detekcia, per-reader derivácia kľúčov, tamper-evident audit) poskytuje merateľne vyššiu odolnosť voči testovaným triedam útokov v porovnaní so štandardným nasadením AEAD. Obmedzenia sú väčšinou dôsledkom vedomého zúženia scope na účely diplomovej práce a sú jasne dokumentované, čo umožňuje ich adresovanie v budúcich iteráciách.

7 Zoznam použitej literatúry

- [1] Elaine Barker. NIST SP 800-57 Part 1 Rev. 5: Recommendation for Key Management – Part 1: General. NIST, 2020. Available from: <https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final>.
- [2] Daniel J. Bernstein. The salsa20 family of stream ciphers. In *New Stream*

- Cipher Designs*, volume 4986 of *Lecture Notes in Computer Science*, pages 84–97. Springer, 2008.
- [3] Frank Denis. The Sodium cryptography library (libsodium). Online documentation, 2024. Available from: <https://doc.libsodium.org/>.
- [4] Espressif Systems. ESP32 Technical Reference Manual, Version 5.3. Espressif Systems, 2024. Available from: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf.
- [5] Flavio D. Garcia, Gerhard de Koning Gans, Ruben Muijers, Peter van Rossum, Roel Verdult, Ronny Wichers Schreur, and Bart Jacobs. Dismantling MIFARE classic. In *Proceedings of the 13th European Symposium on Research in Computer Security (ESORICS)*, volume 5283 of *Lecture Notes in Computer Science*, pages 97–114, 2008.
- [6] Shawn Hernan, Scott Lambert, Tomasz Ostwald, and Adam Shostack. Threat modeling — uncover security design flaws using the STRIDE approach. *MSDN Magazine*, November 2006.
- [7] D. Richard Hipp. SQLite: Security. SQLite Consortium, 2024. Available from: <https://www.sqlite.org/security.html>.
- [8] International Organization for Standardization. ISO/IEC 14443:2018 Identification cards — Contactless integrated circuit cards — Proximity cards. ISO, 2018.
- [9] Hugo Krawczyk and Phil Eronen. RFC 5869: HMAC-based Extract-and-Expand Key Derivation Function (HKDF). IETF, 2010. Available from: <https://datatracker.ietf.org/doc/html/rfc5869>.

- [10] David McGrew. RFC 5116: An Interface and Algorithms for Authenticated Encryption. RFC Editor, 2008. Available from: <https://www.rfc-editor.org/rfc/rfc5116.html>.
- [11] National Institute of Standards and Technology. FIPS 198-1: The Keyed-Hash Message Authentication Code (HMAC). NIST, 2008. Available from: <https://csrc.nist.gov/pubs/fips/198-1/final>.
- [12] National Security Agency. Defense in Depth: A practical strategy for achieving Information Assurance in today's highly networked environments. NSA/IAD, 2012. Available from: <https://apps.nsa.gov/iaarchive/library/ia-guidance/archive/defense-in-depth.cfm>.
- [13] Yoav Nir. RFC 8439: ChaCha20 and Poly1305 for IETF Protocols. RFC Editor, 2018. Available from: <https://www.rfc-editor.org/rfc/rfc8439.html>.
- [14] Private Internet Access and NCC Group. Libsodium Security Assessment. Private Internet Access, 2017. Available from: <https://www.privateinternetaccess.com/blog/libodium-v1-0-12-and-target-v1-0-13-security-assessment/>.
- [15] Eric Rescorla. RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3. RFC Editor, 2018. Available from: <https://www.rfc-editor.org/rfc/rfc8446.html>.
- [16] Karen Scarfone, Wayne Jansen, and Miles Tracy. NIST SP 800-94: Guide to Intrusion Detection and Prevention Systems (IDPS). NIST, 2007. Available from: <https://csrc.nist.gov/pubs/sp/800/94/final>.
- [17] Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2):159–176, 1999. DOI: 10.1145/317087.317089.

8 Zoznam príloh

Priložené elektronické médium obsahuje vetvu `analytical` Git repozitára so štruktúrou: deväť produkčných modulov (`crypto_lib`, `protocol_lib`, `key_manager`, `access_core`, `access_decision`, `access_storage`, `config_loader`, `runtime_events`, `access_admin` — popísané v tabuľke 4–1), `hw_layer/` (firmvér ESP32), `web/` (webové rozhranie), `experiments/` (C++ harness), `analysis/` (Python pipeline), `config/` (YAML konfigurácie), `cmake_other/` (CMake FetchContent), `Dockerfile`, koreňový `CMakeLists.txt` a adresár `tukedip_output/` s textom diplomovej práce a systémovej príručky v L^AT_EX.

Každý modul má podadresár `include/` (verejné hlavičky), `src/` (implementácia) a voliteľne `tests/` (unit testy). Zdrojové súbory používajú Doxygen-štýl komentárov (`///param, @return); identifikácia autora a projektu je zabezpečená cez Git históriu a koreňový README.md. Produkčný kód je vo vetvách main aj analytical identický — vetva main len neobsahuje experiments/, analysis/ a tukedip_output/.`